

Aula 04 - Micro-livraria

Relatório das partes 1 e 2 | DevOps | Execução em 28/08/2026

A atividade foi dividida em duas etapas: adicionar a busca de livro por ID e colocar o serviço de frete em um container. O código ficou na branch **feat/aula04** do repositório usado nas aulas anteriores, sem fazer merge na main. Essa foi a adaptação adotada no lugar do fork separado previsto no roteiro.

```
git remote -v
git branch --show-current
node --version
npm.cmd --version
```

Esses comandos conferem o repositório, a branch e as versões das ferramentas. O ambiente utilizado foi Windows, com Node 22.17.0 e npm 10.9.2.

Registro de execução

Micro-livraria - Aula 04

```
git -c safe.directory='C:/Users/luizf/OneDrive/Área de Trabalho/DevOps' remote -v; git -c
safe.directory='C:/Users/luizf/OneDrive/Área de Trabalho/DevOps' branch --show-current; node --version;
npm.cmd --version
```

```
origin https://github.com/lfbpaiva/DevOps.git (fetch)
origin https://github.com/lfbpaiva/DevOps.git (push)
feat/aula04
v22.17.0
10.9.2
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

```
npm.cmd install
```

O comando instala as dependências do projeto. Foram adicionados 299 pacotes. A instalação terminou, mas apontou 30 vulnerabilidades nas dependências antigas. Não foi usado audit fix --force para evitar alterações incompatíveis com o exercício.

Registro de execução

Micro-livraria - Aula 04

```
npm.cmd install
```

```
npm warn old lockfile
npm warn old lockfile The package-lock.json file was created with an old version of npm,
npm warn old lockfile so supplemental metadata must be fetched from the registry.
npm warn old lockfile
npm warn old lockfile This is a one-time fix-up, please be patient...
npm warn old lockfile
```

```
added 299 packages, and audited 300 packages in 20s
```

```
36 packages are looking for funding
  run `npm fund` for details
```

```
30 vulnerabilities (5 low, 7 moderate, 15 high, 3 critical)
```

```
To address issues that do not require attention, run:
  npm audit fix
```

```
To address all issues (including breaking changes), run:
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.

Os prints dos comandos exibem saídas salvas do PowerShell em um painel de registro. As telas da livraria foram capturadas no navegador. Saídas longas aparecem em trechos identificados.

Parte 1 - Iniciando a aplicação

```
npm.cmd run start
```

Na primeira parte, esse comando inicia os quatro serviços localmente: frontend, controller, inventory e shipping. Os logs mostraram cada um em sua porta. O aviso de atualização não impediu a execução.

Registro de execução

Micro-livraria - Aula 04

```
npm.cmd run start
```

```
Inventory Service running at http://127.0.0.1:3002
Shipping Service running at http://127.0.0.1:3001
Controller Service running on http://127.0.0.1:3000
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
```

Saída capturada do comando executado em PowerShell. Trecho 3/3.

```
curl.exe -sS -i http://localhost:3000/products
```

A requisição GET consulta os livros disponíveis. A opção -i mostra também os cabeçalhos HTTP; -sS esconde a barra de progresso e mantém os erros visíveis. A resposta foi **200 OK** e trouxe os três livros do catálogo.

Registro de execução

Micro-livraria - Aula 04

```
curl.exe -sS -i http://localhost:3000/products
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 437
ETag: W/"1b5-H/SdsXWVDtoSiWQ7y1hB40Z8ZQk"
Date: Fri, 28 Aug 2026 19:42:18 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
[{"id":1,"name":"Refactoring","quantity":10,"price":79.91999816894531,"photo":"/img/refactoring.png","author":"Martin Fowler"}, {"id":2,"name":"Engenharia De Software Moderna","quantity":10,"price":65.9000015258789,"photo":"/img/esm.png","author":"Marco Tulio Valente"}, {"id":3,"name":"Design Patterns","quantity":10,"price":67.98999786376953,"photo":"/img/design.png","author":"Erich Gamma, John Vlissides, Richard Helm, Ralph Johnson"}]
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Parte 1 - Busca de livro por ID

A nova operação foi declarada em **proto/inventory.proto**. Ela recebe uma mensagem Payload com o ID e retorna um ProductResponse, que já existia no contrato.

```
Get-Content proto/inventory.proto
```

Get-Content mostra o conteúdo do arquivo. No trecho abaixo aparecem a operação SearchProductByID e o campo id da mensagem Payload.

Registro de execução

Micro-livraria - Aula 04

```
Get-Content proto/inventory.proto
```

```
syntax = "proto3";

service InventoryService {
  rpc SearchAllProducts(Empty) returns (ProductsResponse) {}
  rpc SearchProductByID(Payload) returns (ProductResponse) {}
}

message Payload {
  int32 id = 1;
}

message Empty{}

message ProductResponse {
  int32 id = 1;
  string name = 2;
  int32 quantity = 3;
  float price = 4;
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.

Em services/inventory/index.js, a busca usa find para localizar o livro pelo ID recebido. Em services/controller/index.js, a rota /product/:id chama esse método e devolve o resultado em JSON.

```
curl.exe -sS -i http://localhost:3000/product/1
```

Esse comando testa a rota nova. A API respondeu **200 OK** com apenas o livro Refactoring, de Martin Fowler, identificado pelo ID 1. A busca ficou no backend, como pede o roteiro.

Registro de execução

Micro-livraria - Aula 04

```
curl.exe -sS -i http://localhost:3000/product/1
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 125
ETag: W/"7d-zkO9ouY3ZEomyWSW8hFOD+NwkY"
Date: Fri, 28 Aug 2026 19:42:20 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"name":"Refactoring","quantity":10,"price":79.91999816894531,"photo":"/img/refactoring.png","author":"Martin Fowler"}
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Parte 1 - Conferindo pelo navegador

Abrir `http://localhost:5000`

O frontend carregou os três livros, com imagens, preços e botões. Isso mostra que a tela conseguiu consultar o catálogo pela API.

Micro Livraria

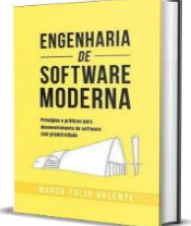
Uma simples livraria utilizando microservices



R\$79.92

Disponível em estoque: 5

Refactoring
Martin Fowler



R\$65.90

Disponível em estoque: 5

Engenharia De Software Moderna
Marco Tulio Valente



R\$67.99

Disponível em estoque: 5

Design Patterns
Erich Gamma, John Vlissides,
Richard Helm, Ralph Johnson

Feito com ❤ na UFMG

Também foram testados os botões de frete e compra. Com o CEP 85810000, a consulta local mostrou R\$ 98,70. O botão Comprar exibiu a mensagem de sucesso. A compra é apenas uma simulação visual: não há pagamento nem gravação de pedido. O frete é aleatório e não consulta uma transportadora.

O estoque exibido nos cartões é um texto fixo do exemplo original. Por isso, ele não representa uma atualização de estoque após clicar em Comprar.

Parte 1 - Testes e envio

```
node --test tests/livraria.test.cjs
```

O runner de testes do Node consulta a aplicação em execução. A suíte verifica a listagem, a busca dos IDs 1, 2 e 3 e a consulta de frete. Os **cinco testes passaram**. Os casos de busca usam os IDs existentes no catálogo.

Registro de execução

Micro-livraria - Aula 04

```
node --test tests/livraria.test.cjs
```

```
...
# Subtest: busca o livro 3 pelo identificador
ok 4 - busca o livro 3 pelo identificador
---
  duration_ms: 6.3387
  type: 'test'
...
# Subtest: consulta o frete pelo controller
ok 5 - consulta o frete pelo controller
---
  duration_ms: 6.6479
  type: 'test'
...
1..5
# tests 5
# suites 0
# pass 5
# fail 0
```

Saída capturada do comando executado em PowerShell. Trecho 2/3.

```
git add .gitignore package.json package-lock.json LICENSE proto services tests/livraria.test.cjs
git commit -m "Implementa busca de livros por ID"
git push -u origin feat/aula04
```

git add prepara os arquivos, commit registra a alteração e push envia a branch ao GitHub. A opção -u associa a branch local à remota. A parte 1 ficou no commit **4a6588e**.

Registro de execução

Micro-livraria - Aula 04

```
git push -u origin feat/aula04
```

```
remote:
remote: Create a pull request for 'feat/aula04' on GitHub by visiting:
remote:   https://github.com/lfbpaiva/DevOps/pull/new/feat/aula04
remote:
branch 'feat/aula04' set up to track 'origin/feat/aula04'.
To https://github.com/lfbpaiva/DevOps.git
 * [new branch]   feat/aula04 -> feat/aula04
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Parte 2 - Docker e Dockerfile

```
docker version
docker compose version
```

docker version verifica o cliente e o motor de containers. Ambos responderam com a versão 29.2.1. O Compose instalado era 5.1.0, embora esta tarefa use build e run diretamente.

Registro de execução Micro-livraria - Aula 04

```
docker version; docker compose version
```

Client:

```
Version:      29.2.1
API version:  1.53
Go version:   go1.25.6
Git commit:   a5c7197
Built:        Mon Feb  2 17:20:16 2026
OS/Arch:      windows/amd64
Context:      desktop-linux
```

Server: Docker Desktop 4.65.0 (221669)

Engine:

```
Version:      29.2.1
API version:  1.53 (minimum version 1.44)
Go version:   go1.25.6
Git commit:   6bc6209
Built:        Mon Feb  2 17:17:24 2026
OS/Arch:      linux/amd64
Experimental: false
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.

Get-Content shipping.Dockerfile

O Dockerfile usa **node:22-alpine** como base e /app como diretório de trabalho. COPY leva os arquivos para a imagem, RUN instala as dependências e CMD inicia o serviço. Foi usado npm ci --omit=dev para seguir o lockfile e deixar de fora ferramentas de desenvolvimento.

Registro de execução Micro-livraria - Aula 04

```
Get-Content shipping.Dockerfile; Get-Content .dockerignore; node -p "require('./package.json').scripts.start"
```

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY proto/shipping.proto ./proto/shipping.proto
COPY services/shipping/index.js ./services/shipping/index.js
COPY LICENSE ./LICENSE
CMD ["node", "/app/services/shipping/index.js"]
node_modules/
.git/
.venv/
__pycache__/
.pytest_cache/
.ruff_cache/
.coverage
.env
*.log
*.pdf
```

Saída capturada do comando executado em PowerShell. Trecho 1/2.

O .dockerignore exclui node_modules, Git, ambiente Python, logs, .env e PDFs. A imagem copia apenas as dependências, o contrato de frete, o código do Shipping e a licença.

Parte 2 - Construindo a imagem

```
docker build --progress=plain -t micro-livraria/shipping -f shipping.Dockerfile ./
```

build constrói a imagem. -t define o nome, -f seleciona o Dockerfile e ./ indica a pasta usada como contexto. --progress=plain mantém o log em texto.

A construção terminou com a imagem **micro-livraria/shipping:latest** criada. Na instalação de produção, o npm apontou 11 vulnerabilidades. O build ter passado não elimina esses alertas.

Registro de execução

Micro-livraria - Aula 04

```
docker build --progress=plain -t micro-livraria/shipping -f shipping.Dockerfile ./
```

```
#10 DONE 0.0s

#11 [7/7] COPY LICENSE ./LICENSE
#11 DONE 0.0s

#12 exporting to image
#12 exporting layers
#12 exporting layers 0.9s done
#12 exporting manifest sha256:8bee960b8aed131785c2e72791f06222cf7e844415125d8406db510b87430a09 0.0s done
#12 exporting config sha256:090f634915a1ca8ebacce196d6f59511f01a7fc8916e74d715375acfe32e2d96 0.0s done
#12 exporting attestation manifest sha256:67e2b3bab78c5fd69d50106b75c16c1f7544ea4452691d1406cf73da29986368 0.0s done
#12 exporting manifest list sha256:b41d209cdf8e08015ef55bab4ec8f8891dc3d0cd9048977fd092cc26ab8af331
#12 exporting manifest list sha256:b41d209cdf8e08015ef55bab4ec8f8891dc3d0cd9048977fd092cc26ab8af331 0.0s done
#12 naming to docker.io/micro-livraria/shipping:latest done
#12 unpacking to docker.io/micro-livraria/shipping:latest
#12 unpacking to docker.io/micro-livraria/shipping:latest 0.9s done
#12 DONE 1.9s
```

Saída capturada do comando executado em PowerShell. Trecho 4/4.

```
npm.cmd run start
```

Antes de executar novamente, start-shipping foi removido do script start e os processos da parte 1 foram encerrados. Agora o comando inicia somente frontend, inventory e controller. O frete será atendido pelo container, evitando conflito na porta 3001.

Registro de execução

Micro-livraria - Aula 04

```
npm.cmd run start
```

```
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node services/inventory/index.js`
[nodemon] 2.0.7
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node services/controller/index.js`
WARNING: Checking for updates failed (use `--debug` to see full error)
INFO: Accepting connections at http://localhost:5000
Inventory Service running at http://127.0.0.1:3002
Controller Service running on http://127.0.0.1:3000
```

Saída capturada do comando executado em PowerShell. Trecho 2/2.

Parte 2 - Executando e testando o container

```
docker run -dit --name shipping -p 127.0.0.1:3001:3001 micro-livraria/shipping
docker ps --filter name=shipping
docker logs shipping
```

run cria o container. A opção -d o deixa em segundo plano; -i e -t habilitam entrada e terminal. --name define o nome shipping, e -p publica a porta 3001 somente no computador local. ps consulta o estado; logs mostra a saída do serviço.

O container ficou em estado **Up**, e o log mostrou Shipping Service running. O mapeamento liga a porta 3001 do computador à porta 3001 do container.

Registro de execução

Micro-livraria · Aula 04

```
docker ps --filter name=shipping; docker logs shipping
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
2205e71c5c55	micro-livraria/shipping	"docker-entrypoint.s..."	1 second ago	Up 1 second	127.0.0.1:3001->3001/tcp shipping

Shipping Service running at http://127.0.0.1:3001

Saída capturada do comando executado em PowerShell. Trecho 1/1.

```
curl.exe -sS -i http://localhost:3000/shipping/85810000
```

A chamada entra no Controller local, que consulta o Shipping no Docker. A resposta foi **200 OK**, com o CEP informado e um valor de frete. Como o Shipping local não estava mais rodando, o retorno veio do serviço no container.

Registro de execução

Micro-livraria · Aula 04

```
curl.exe -sS -i http://localhost:3000/shipping/85810000
```

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 44
ETag: W/"2c-fMcOe02sOT79WfnwFgJCnFThDAg"
Date: Fri, 28 Aug 2026 19:52:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5

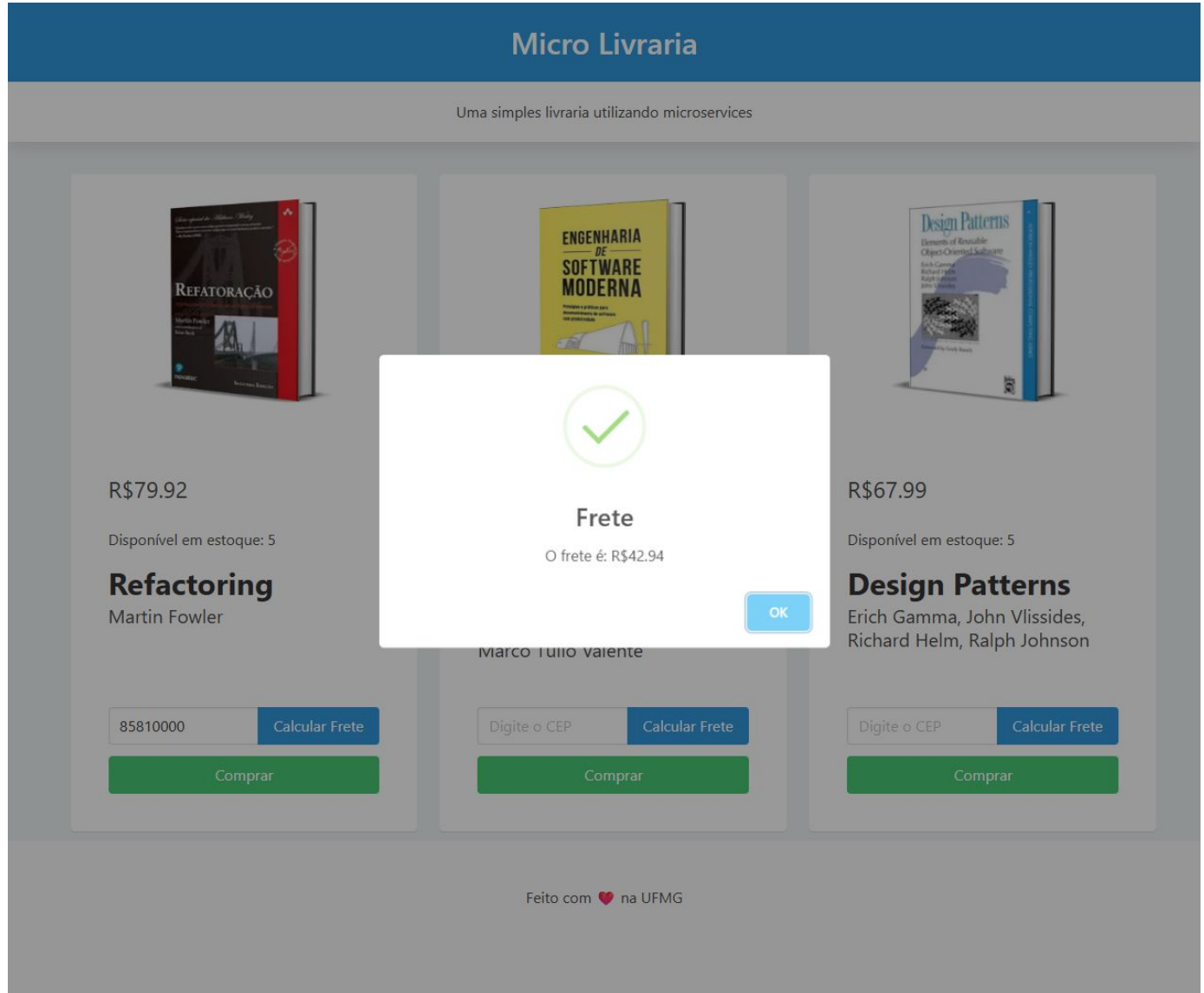
{"cep":"85810000","value":35.09730529785156}
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Parte 2 - Frete na interface

```
Recarregar http://localhost:5000  
Informar CEP 85810000 > Calcular Frete
```

Depois da troca para o Docker, a mesma tela continuou funcionando. Nesta consulta, o frete mostrado foi **R\$ 42,94**.



O valor mudou em relação ao teste pelo curl porque foram feitas chamadas diferentes e o exemplo gera um número aleatório. Não foi necessário alterar o frontend para usar o frete no container.

Parte 2 - Testes e publicação

```
node --test tests/livraria.test.cjs
```

A mesma suíte da parte 1 foi executada com o frete no Docker. Os **cinco testes passaram novamente**. Assim, a listagem, as buscas por ID e a consulta de frete mantiveram o comportamento após a mudança.

Registro de execução

Micro-livraria · Aula 04

```
node --test tests/livraria.test.cjs

...
# Subtest: busca o livro 3 pelo identificador
ok 4 - busca o livro 3 pelo identificador
...
  duration_ms: 4.3279
  type: 'test'
...
# Subtest: consulta o frete pelo controller
ok 5 - consulta o frete pelo controller
...
  duration_ms: 6.2809
  type: 'test'
...
1..5
# tests 5
# suites 0
# pass 5
# fail 0
```

Saída capturada do comando executado em PowerShell. Trecho 2/3.

```
git add .dockerignore shipping.Dockerfile package.json .github/workflows/ci.yml
git commit -m "Adiciona container para o servico de frete"
git push origin feat/aula04
```

Os arquivos da segunda parte foram registrados e enviados à mesma branch. O commit é **f02b3b0**. O workflow também recebeu um job que constrói a imagem, inicia os serviços e roda os testes de integração.

Registro de execução

Micro-livraria · Aula 04

```
git commit -m "Adiciona container para o servico de frete"

[feat/aula04 f02b3b0] Adiciona container para o servico de frete
4 files changed, 56 insertions(+), 1 deletion(-)
create mode 100644 .dockerignore
create mode 100644 shipping.Dockerfile
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Registro de execução

Micro-livraria · Aula 04

```
git push origin feat/aula04

To https://github.com/lfbpaiva/DevOps.git
4a6588e..f02b3b0  feat/aula04 -> feat/aula04
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Parte 2 - Limpeza do ambiente

```
docker stop shipping
docker rm shipping
```

stop encerra o container; rm remove a instância parada. Os dois comandos retornaram shipping, indicando que a operação foi concluída.

Registro de execução

Micro-livraria - Aula 04

```
docker stop shipping
```

```
shipping
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Registro de execução

Micro-livraria - Aula 04

```
docker rm shipping
```

```
shipping
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

```
docker rmi micro-livraria/shipping
```

rmi remove a imagem. A saída mostrou Untagged e Deleted. Foram removidos somente o container e a imagem criados para esta atividade.

Registro de execução

Micro-livraria - Aula 04

```
docker rmi micro-livraria/shipping
```

```
Untagged: micro-livraria/shipping:latest
```

```
Deleted: sha256:b41d209cdf8e08015ef55bab4ec8f8891dc3d0cd9048977fd092cc26ab8af331
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

```
docker ps -a --filter name=shipping
docker image ls micro-livraria/shipping
```

As listagens finais não mostraram o container nem a imagem. Para repetir o teste, será preciso executar build e run novamente. A imagem base e o cache de construção podem permanecer no Docker.

Registro de execução

Micro-livraria - Aula 04

```
docker ps -a --filter name=shipping; docker image ls micro-livraria/shipping
```

```
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS          NAMES
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
IMAGE         ID          DISK USAGE  CONTENT SIZE  EXTRA
```

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Conferência final

```
gh run view 33205879015 --repo lfbpaiva/DevOps
```

O comando consulta a execução no GitHub Actions. Os jobs **testes-pytest** e **testes-livraria-docker** passaram. Apareceram avisos sobre versões antigas de algumas Actions, mas não houve falha nos testes.

Registro de execução

Micro-livraria · Aula 04

```
gh run view 33205879015 --repo lfbpaiva/DevOps
```

✓ feat/aula04 CI · 33205879015

Triggered via push about 1 minute ago

JOBS

✓ testes-pytest in 13s (ID 98966663552)

✓ testes-livraria-docker in 22s (ID 98966663768)

ANNOTATIONS

! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>

testes-pytest: .github#2

! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-node@v4. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>

testes-livraria-docker: .github#4

For more information about a job, try: `gh run view --job=<job-id>`

View this run on GitHub: <https://github.com/lfbpaiva/DevOps/actions/runs/33205879015>

Saída capturada do comando executado em PowerShell. Trecho 1/1.

Com isso, as duas partes foram executadas: a busca por ID retornou o livro solicitado e o frete funcionou em Docker, tanto pela API quanto pela interface. O código continua sendo um exemplo de aula: o frete é aleatório, a compra é simulada e as dependências precisam de revisão antes de qualquer uso em produção.

O Docker apresentou um erro de acesso a sockets temporários antes dos testes. Foi possível iniciá-lo após guardar esses diretórios como backup, sem apagar imagens ou containers existentes.

Roteiro e código original: github.com/aserg-ufmg/micro-livraria

Entrega: github.com/lfbpaiva/DevOps/tree/feat/aula04

CI: github.com/lfbpaiva/DevOps/actions/runs/33205879015

O código original é de Rodrigo Brito, sob orientação de Marco Tulio Valente (ASERG/UFMG). A licença MIT foi preservada; o roteiro informa licença CC-BY. Aula04.pdf foi usado como apoio sobre Docker. Este relatório segue as duas tarefas da micro-livraria indicadas no enunciado.